GCA Session 2 - Lab Report

Stijn Vandendriessche
Master of Electronics and ICT Engineering Technology

Abstract: This report investigates race conditions in the context of maximum value detection, utilizing a C++ application that evaluates the performance of different approaches on both CPU and GPU. Specifically, this study compares the use of atomic functions and the reduction method for determining the maximum value of an array.
Keywords: CUDA, GPU, CPU, C++

1 Design and time complexity

This report employs three distinct techniques: the sequential CPU approach, a GPU kernel with atomic operations, and a reduction-based GPU kernel.

1.1 sequential CPU approach

The sequential approach consists of a simple for loop mechanism that will iterate over every value and check if it is greater than the current maximum. This results in a time complexity of **O(n)**.

1.2 atomic GPU kernel

This method launches a set of concurrent threads to execute in parallel. However, due to the restriction that only one atomic operation can run at a time, the outcome is similar to that of a for loop's execution. The time complexity is also **O(n)**.

1.3 reduction GPU kernel

The reduction technique is a parallel computing strategy designed to tackle the limitation of atomic kernels working almost sequentially. It employs multiple threads, each tasked with processing a distinct portion of the array. By subdividing the problem into smaller pieces, the technique can identify progressively fewer "sub-maxima" at each step, ultimately leading to a single value representing the maximum.

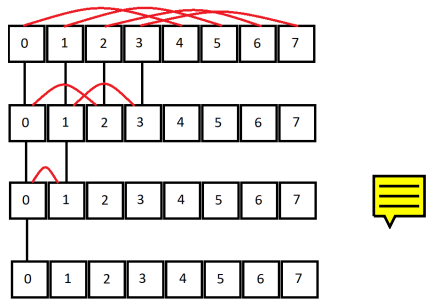


Figure 1: *Visual representation of the reduction technique*

Considering that the number of "sub-maxima" halves at every step while needing to synchronize all threads between these steps, the execution requires log(n) steps. Consequently, the time complexity is **O(log(n))**.

2 limits

The synchronization of the reduction kernel operates at a block level rather than a grid level (for details on these mechanisms, refer to the previous report). This means that the maximum number of threads is capped at 1024, allowing for a maximum of 2048 elements in the array.

The 2048-element limit can be surpassed by dividing the array into smaller chunks that fit within the 1024-thread block size. Each block can then compute its own maximum, and these local maximums can be combined in a final step to find the overall maximum. This approach can also be applied to 2D arrays. However, it's important to note that the additional synchronisation can only happen on the CPU, which increases overhead significantly.

3 results

The three algorithms were executed 1 000 times, with array sizes varying from 2 to 2048. The intriguing outcomes are depicted in the forthcoming graph.

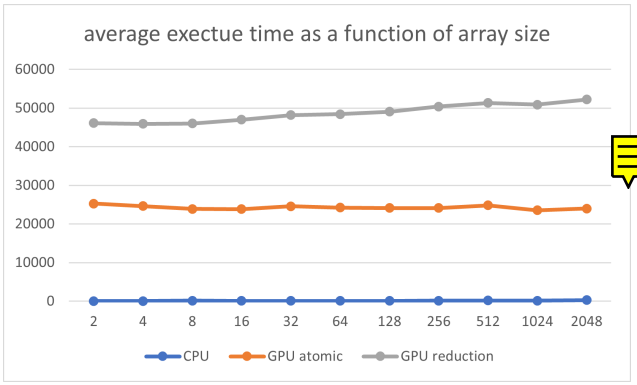


Figure 2: *average execution time as a function of array size*

The results seem surprising, and this could have several reasons.

Firstly, GPUs are generally more efficient for processing large datasets in parallel, but for small input sizes the overhead associated with launching GPU kernels is often not justified. Since the array size ranged from 2 to 2048 (not very large), this could have impacted the results.

Secondly, the data transfer overhead between the CPU and GPU could introduce significant delays, outweighing the benefits of GPU parallelism. These factors together can explain the observed trend where the CPU execution outperformed the GPU kernels, which was unexpected.

4 Github link

`https://github.com/StijnVandendriessche1/GCA_lab2.git`